

Lecture 3

Scheduling

CS 537, Spring 2026
Mike Swift & **Tej Chajed**

Some processes have been waiting since 1997, but the scheduler knows best.

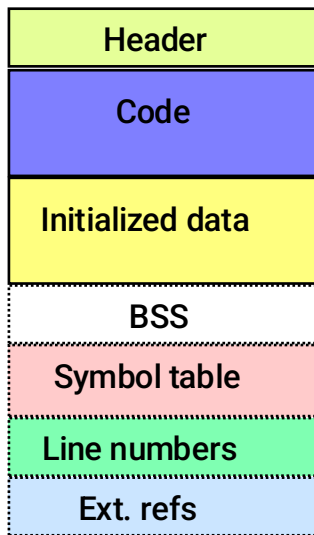


Would you rather fight a:

- horse-sized duck
- 100 duck-sized horses

RECAP

Process vs Program



Executable

Program is passive

- Code + data

Process is running program

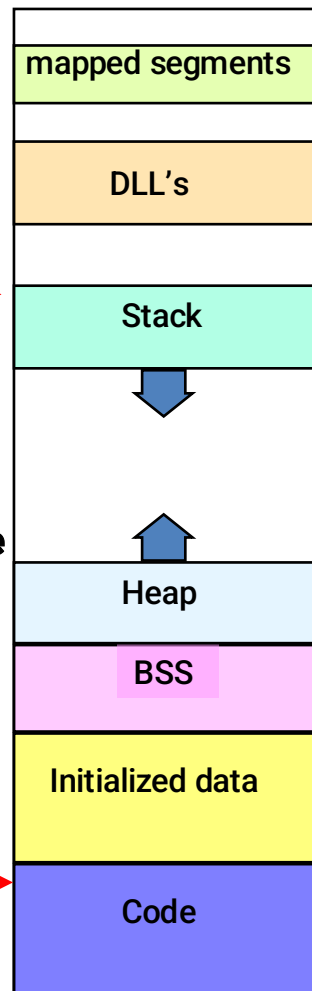
- stack, regs, program counter

Example:

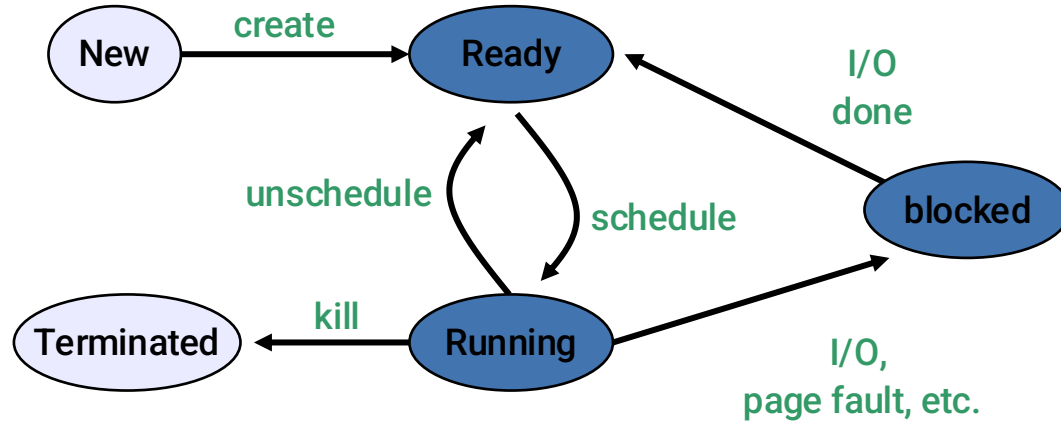
We both run Firefox:

- Same program
- Separate processes

**Process
address space**

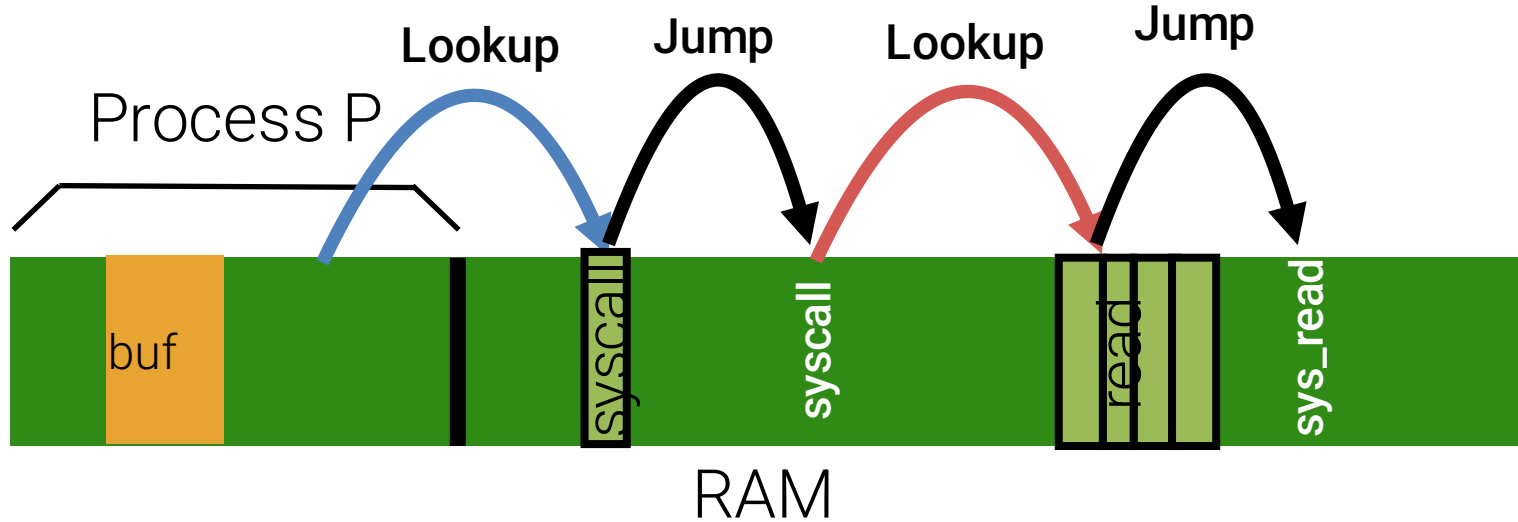


Process state transitions

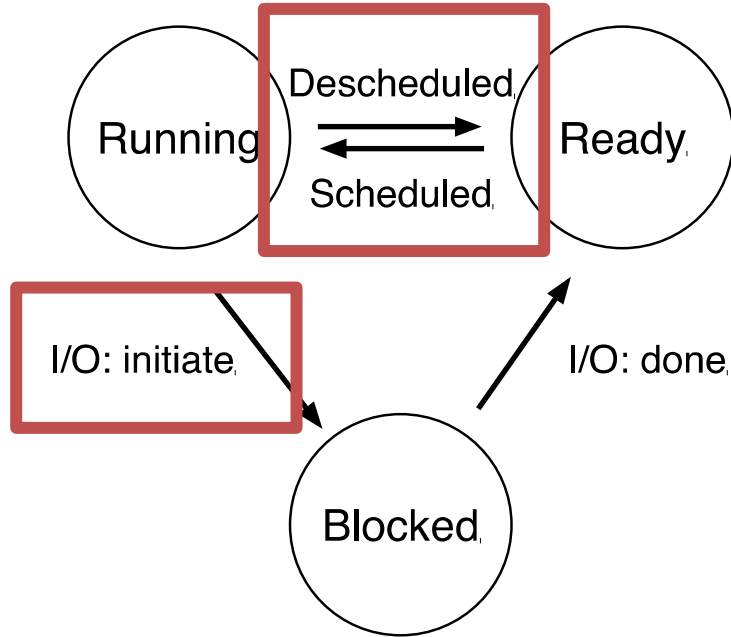


What can cause schedule/unschedule transitions?

System Call



Kernel can access user memory to fill in user buffer
return-from-trap at end to return to Process P



POLICY ?

How should OS decide what process to run?

Learning outcomes

- Compare different scheduling policies based on several metrics
- Articulate the challenges from interactive and batch processes
- Articulate the challenge of scheduling under incomplete info

Vocabulary

Workload: set of jobs (arrival time, run_time)

Job ~ Current execution of a process

- Alternates between CPU and I/O

- Moves between ready and blocked queues

- Also called a “task”

Scheduler: Decides which ready job to run

Metric: measurement of scheduling quality

**Question: what is
the scheduling
metric for a
supermarket?
What is your metric
at the supermarket?**



Supermarket scheduling metrics

Supermarket: maximize throughput
(total people served per hour)

Me: turnaround time
(how fast do I get checked out)

Scheduler goal: avoid starvation

Schedulers typically try to prevent starvation

- **starvation** occurs when a process is prevented from making progress, because another process has a resource it needs

A poor scheduling policy can cause starvation

- e.g., if an important process always prevents a less important process from running on the CPU

APPROACH

Assumptions

Metric



Scheduling
policy

(Initial) ASSUMPTIONS

1. Each job runs for the same amount of time
2. All jobs arrive at the same time
3. All jobs only use the CPU (no I/O)
4. Run-time of each job is known

METRIC 1: Turnaround Time

Turnaround time = *completion_time* - *arrival_time*

Example:

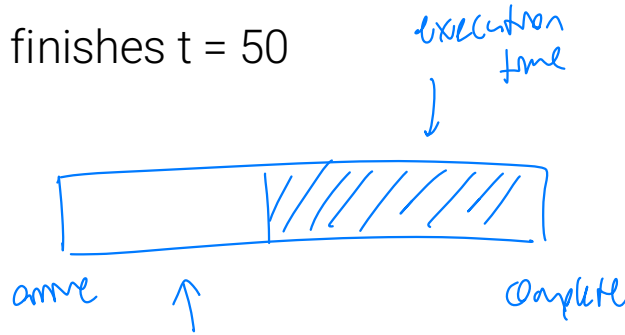
Process A arrives at time $t = 10$, finishes $t = 30$

Process B arrives at time $t = 10$, finishes $t = 50$

Turnaround time

A = 20, B = 40

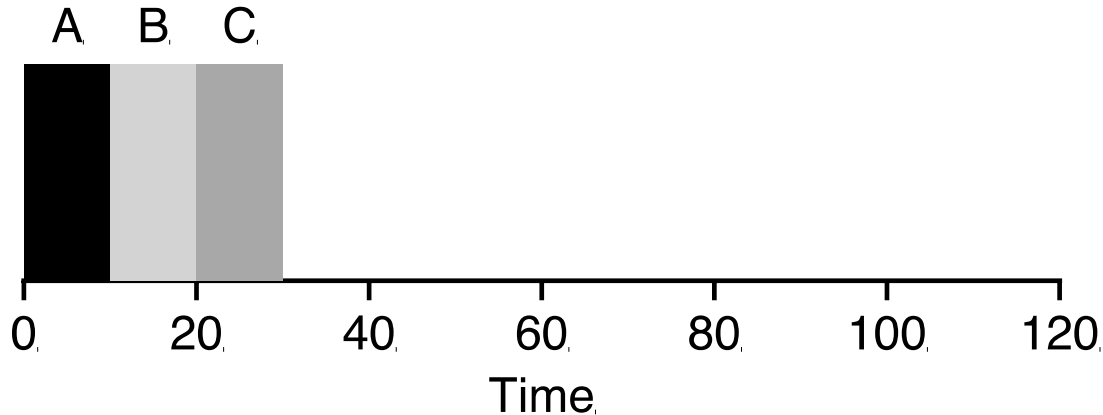
Average = 30



FIFO / FCFS

Job	arrival(s)	run time (s)	turnaround (s)
A	~0	10	10
B	~0	10	20
C	~0	10	30

Run jobs in the order they arrive



Average Turnaround Time?

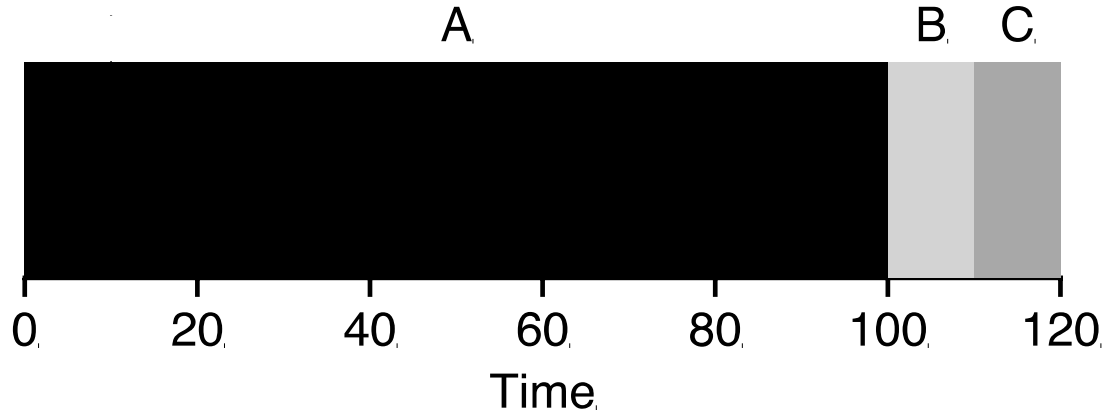
$$= (10+20+30)/3 = 20$$

ASSUMPTIONS

1. Each job runs for the same amount of time
2. All jobs arrive at the same time
3. All jobs only use the CPU (no I/O)
4. Run-time of each job is known

Variable-length jobs

Job	Arrival(s)	run time (s)
A	~0	100
B	~0	10
C	~0	10



Average
Turnaround Time ?

$$= (100 + 110 + 120) / 3 \\ = 110$$

What is one
schedule that could
be better?

CHALLENGE

Turnaround time suffers when short jobs must wait for long jobs

New scheduler:

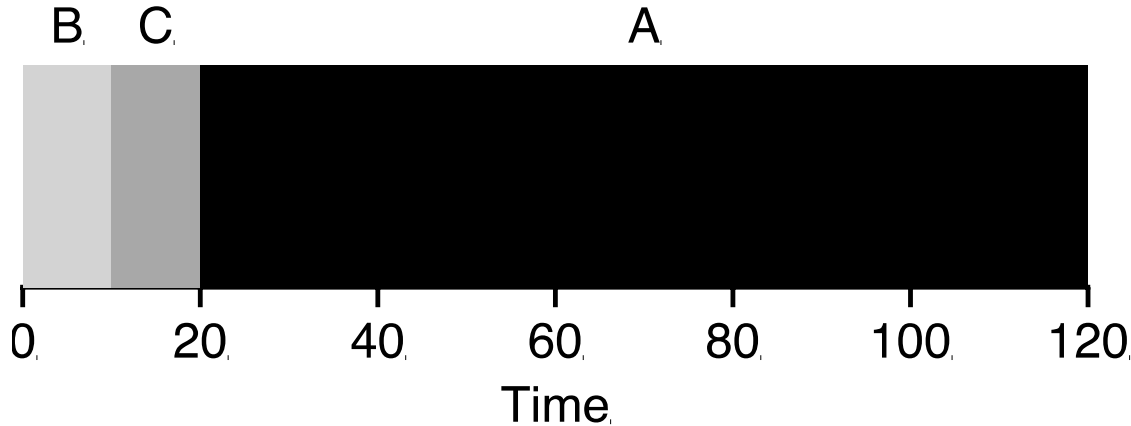
SJF (Shortest Job First)

Choose job with smallest run time!

Shortest Job First (SJF)

Job	Arrival(s)	run time (s)	Turnaround (s)
A	~0	100	120
B	~0	10	10
C	~0	10	20

Run remaining
job with
shortest run
time next



Average Turnaround
Time?

$$= (10 + 20 + 120) / 3 = 50$$

□ □ □ □



What would SJF
mean at a grocery
store?
What could go
wrong?



Head of line blocking



ASSUMPTIONS

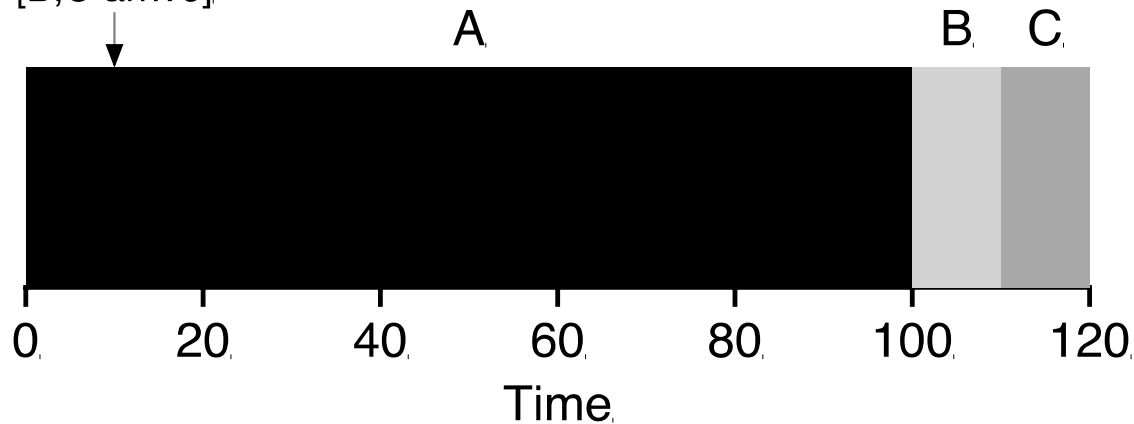
- ~~1. Each job runs for the same amount of time~~
2. All jobs arrive at the same time
3. All jobs only use the CPU (no I/O)
4. Run-time of each job is known

Job	Arrival(s)	run time (s)
A	~0	100
B	10	10
C	10	10

What will be the schedule with SJF?

Job	Arrival(s)	run time (s)
A	~0	100
B	10	10
C	10	10

[B,C arrive]



Average
Turnaround
Time ?

$$\begin{aligned}
 &= (100 + 100 + 110) / 3 \\
 &= 103.3s
 \end{aligned}$$

PREEMPTIVE SCHEDULING

Previous schedulers:

- FIFO and SJF are non-preemptive
- Only schedule new job when previous job voluntarily relinquishes CPU

New idea: preemption *mechanism*

- Schedule different job by taking CPU away from a *running* job

Shortest Time-to-Completion First (STCF)

- Always schedule job with minimum time remaining

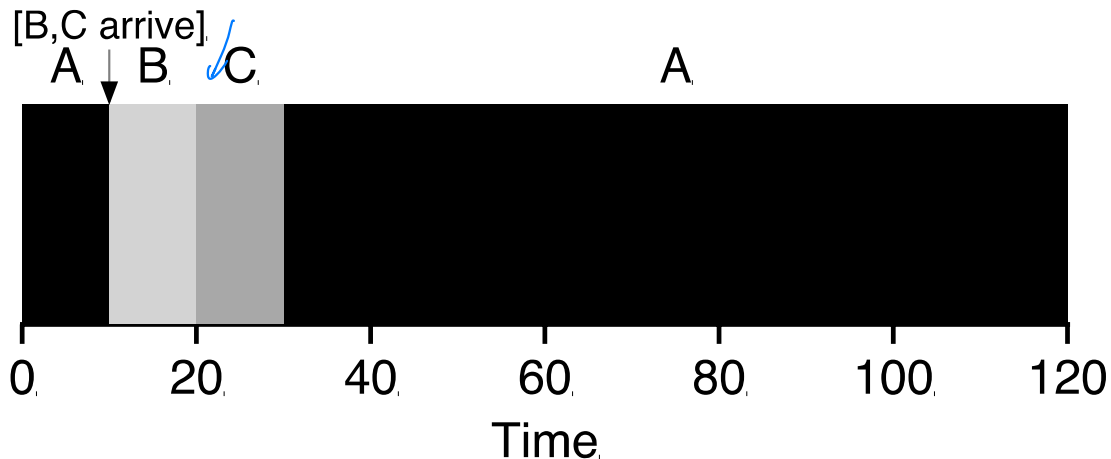
PREEMPTIVE STCF

Job	Arrival(s)	run time (s)
A	~0	100
B	10	10
C	10	10

at time 10:

A remaining time: 90s
B 10s
C 10s

Average
Turnaround
Time



$$= (10 + 20 + 120) / 3$$

$$= 50s$$

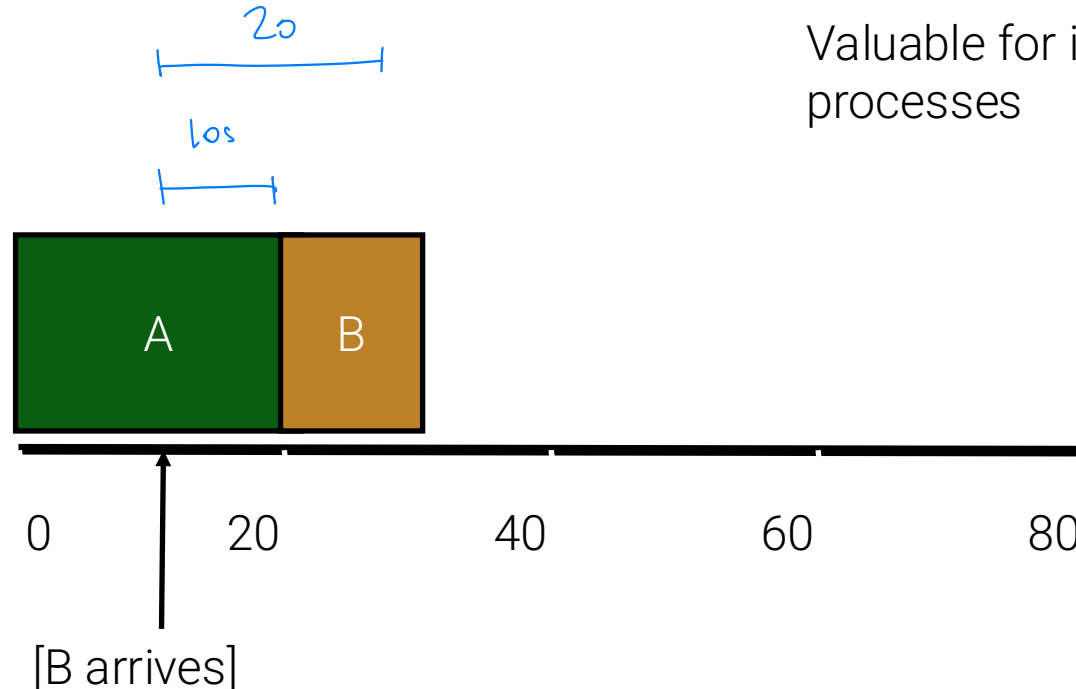
METRIC 2: Response Time

Response time = *first_run_time* - *arrival_time*

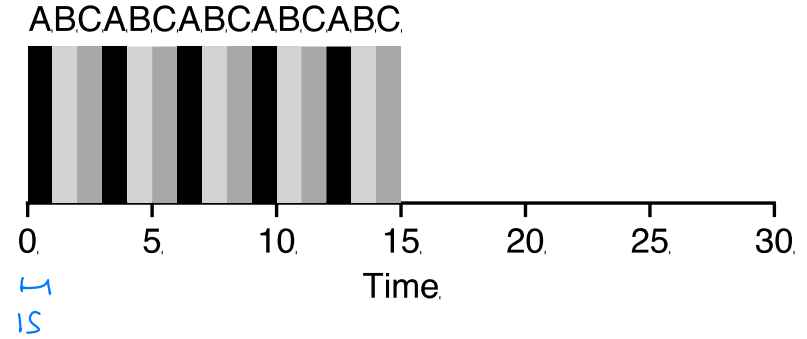
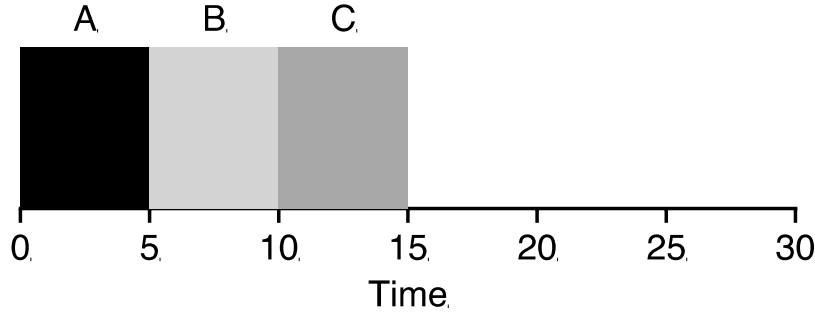
B's turnaround: 20s

B's response: 10s

Valuable for interactive processes

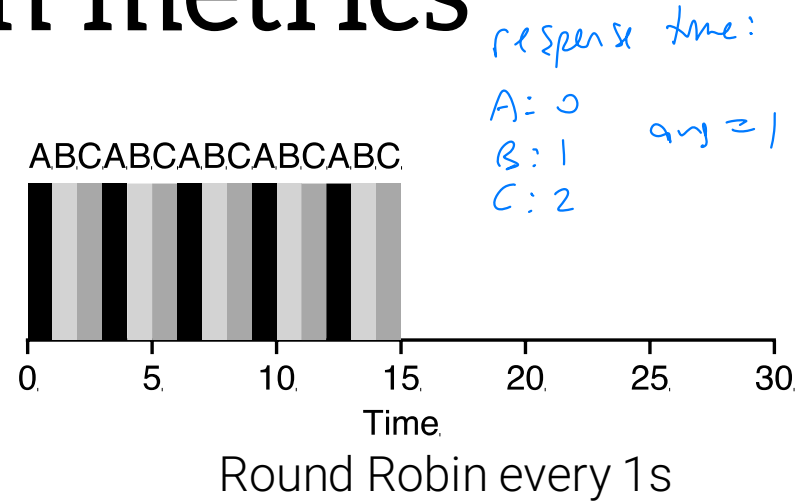
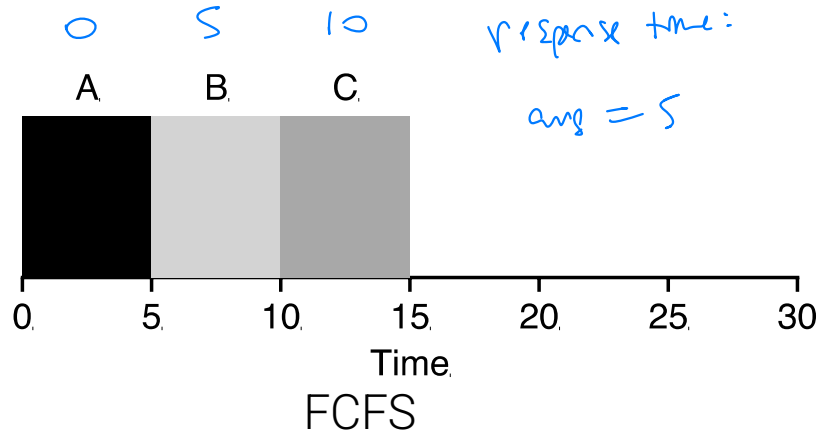


Round Robin Scheduler



Key idea: Switch periodically to reduce response time

Round Robin metrics



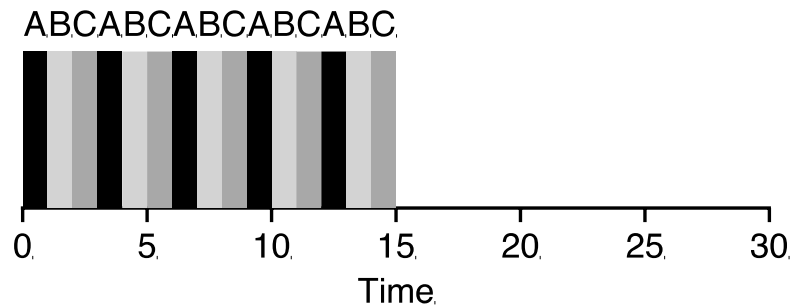
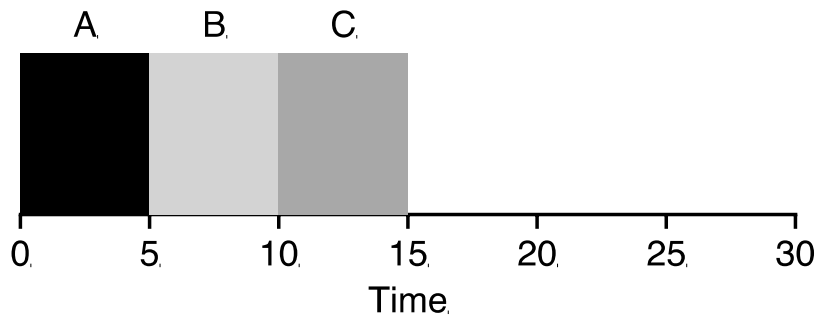
Average Response Time?

A: 5
B: 10
C: 15
avg = 10

Average Turnaround Time?

A: 13
B: 14
C: 15
avg = 14

Round Robin metrics



Average Response Time

$$(0 + 5 + 10)/3 = 5s$$

$$(0 + 1 + 2)/3 = 1s$$

Average Turnaround Time

$$(5 + 10 + 15)/3 = 10s$$

$$(13 + 14 + 15)/3 = 14s$$

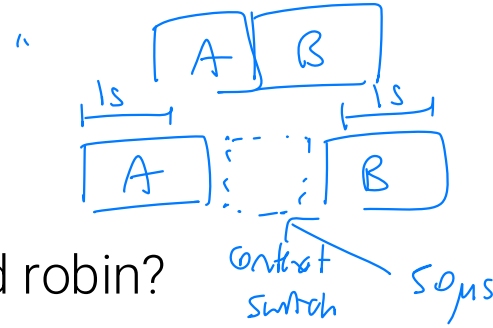
TRADE-OFFS

Round robin increases turnaround time, decreases response time

Tuning challenges:

What is a good time slice for round robin?

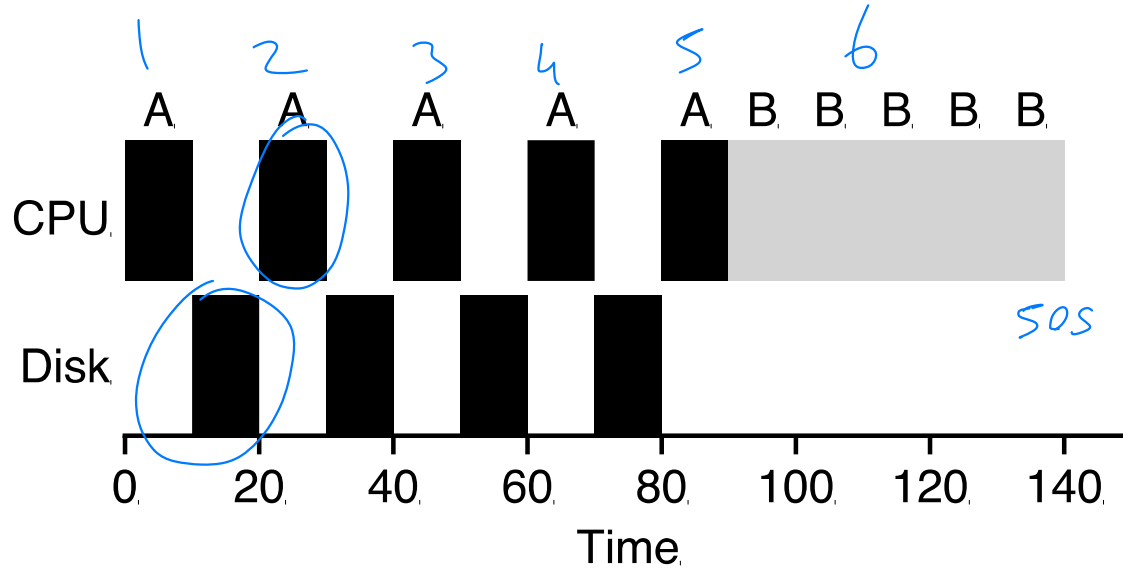
What is the overhead of context switching?



ASSUMPTIONS

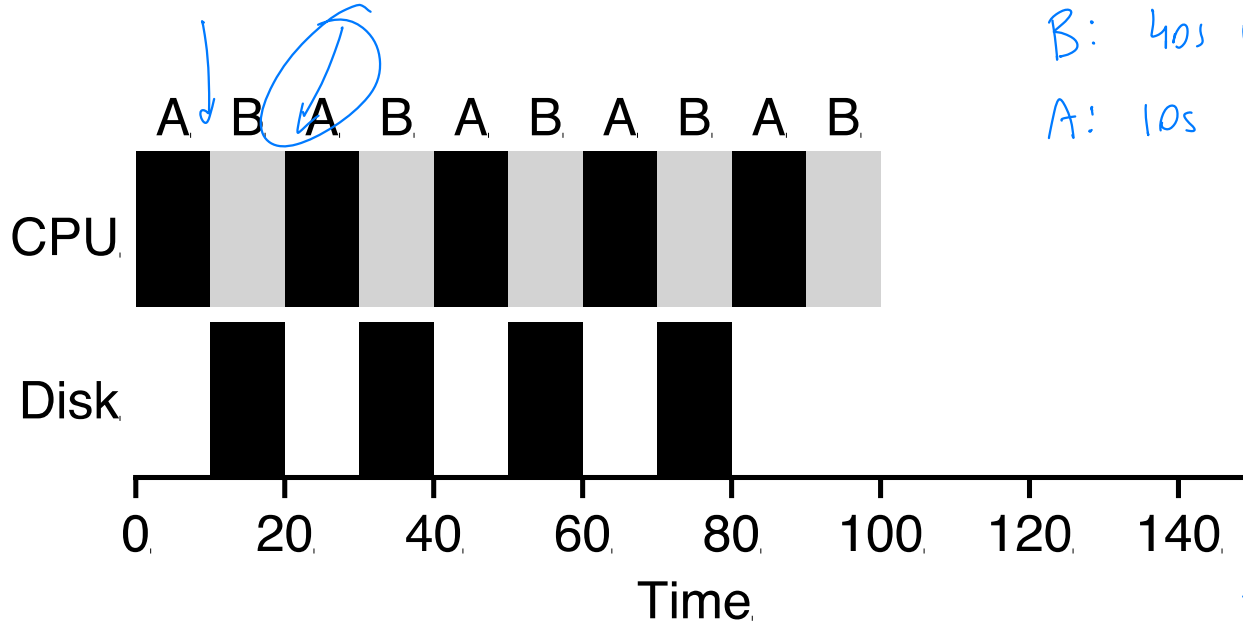
- ~~1. Each job runs for the same amount of time~~
- ~~2. All jobs arrive at the same time~~
3. All jobs only use the CPU (no I/O)
4. Run-time of each job is known

NOT I/O AWARE



Job holds onto CPU while blocked on disk!

I/O-Aware Scheduling



B: 40s remaining

A: 10s CPU burst remaining

Each CPU burst is shorter than Job B

With SCTF, Job A preempts Job B

Treat Job A as separate CPU bursts.
When Job A completes I/O, another Job A is ready



Quiz 3

<https://go.wisc.edu/9wy07s>



Quiz answers

In a uniprocessor system, there can be more than one processes in READY and BLOCKED state while at most one process in RUNNING state.

TRUE

FALSE

A shortest-job-first (SJF) scheduler has the following disadvantages?

- a. We may not know how long a process will take until it's already done running.
- b. If a short process arrives later than a long process, it must wait for the long process to finish.
- c. A and B both.
- d. None of the above.

Quiz Answers

For FIFO/FCFS scheduling policy, what is the average **turnaround time** for the processes if all of them arrive at the same time? "~" indicates just after the time

Job	Arrival(s)	Run time (s)
A	0	5
B	~0	5
C	~0	5

ASSUMPTIONS

- ~~1. Each job runs for the same amount of time~~
- ~~2. All jobs arrive at the same time~~
- ~~3. All jobs only use the CPU (no I/O)~~
4. Run-time of each job is known

MULTI-LEVEL FEEDBACK QUEUE

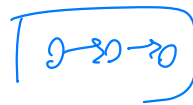
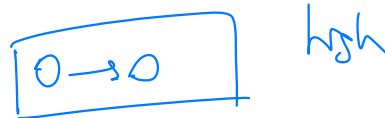
MLFQ: GENERAL PURPOSE SCHEDULER

Must support two job types with distinct goals

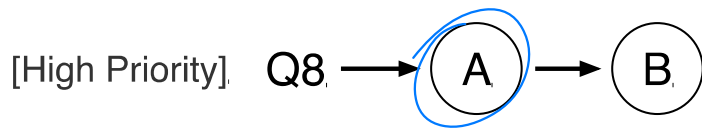
- “interactive” programs care about response time
- “batch” programs care about turnaround time

Approach:

- Multiple levels of round-robin
- Each level has higher priority than lower level
- Can preempt them



MLFQ EXAMPLE



“Multi-level” – Each level is a queue!

Rules for MLFQ

Rule 1: If $\text{priority}(A) > \text{priority}(B)$
A runs

Rule 2: If $\text{priority}(A) == \text{priority}(B)$,
A & B run in RR

CHALLENGES

How to set priority?

What do we do when a new process arrives?

Does a process stay in one queue or move between queues?

Approach:

Use past behavior of process to predict future!

Guess how CPU burst (job) will behave based on past CPU bursts

MORE MLFQ RULES

Rule 1: If $\text{priority}(A) > \text{priority}(B)$, A runs

Rule 2: If $\text{priority}(A) == \text{priority}(B)$, A & B run in RR

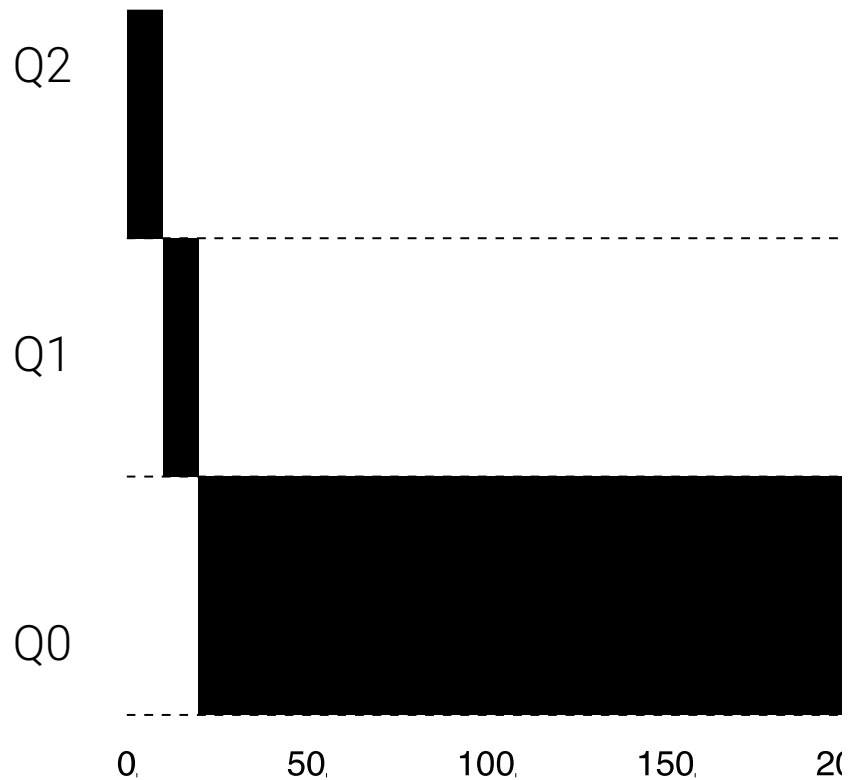
Rule 3: Processes start at top priority

Rule 4: If job uses whole slice, demote process

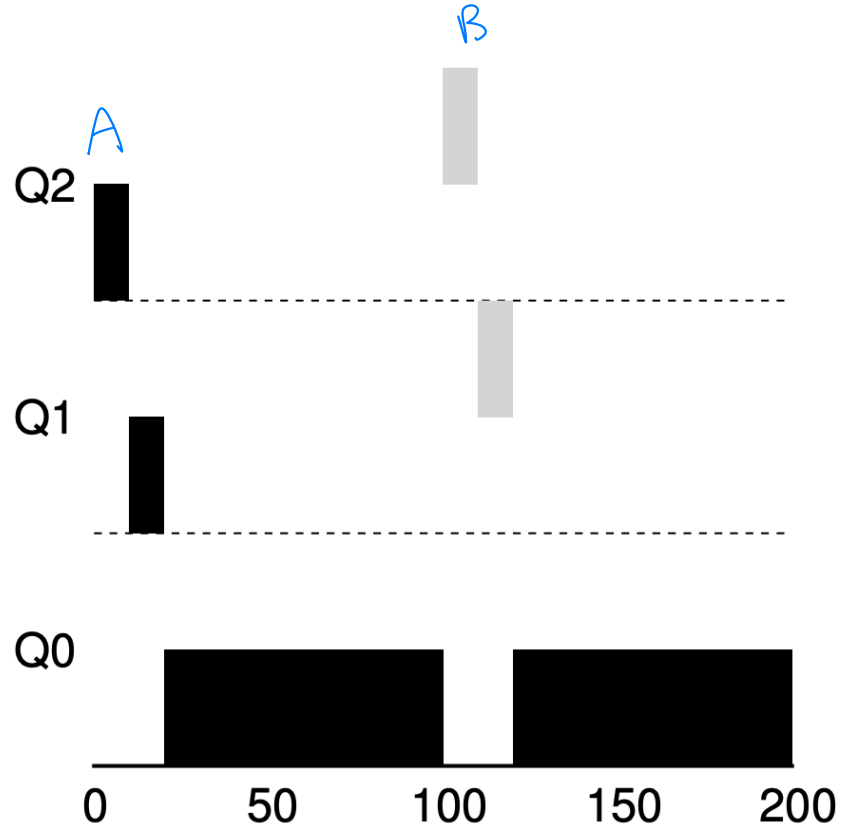
feedback

(longer time slices at lower priorities)

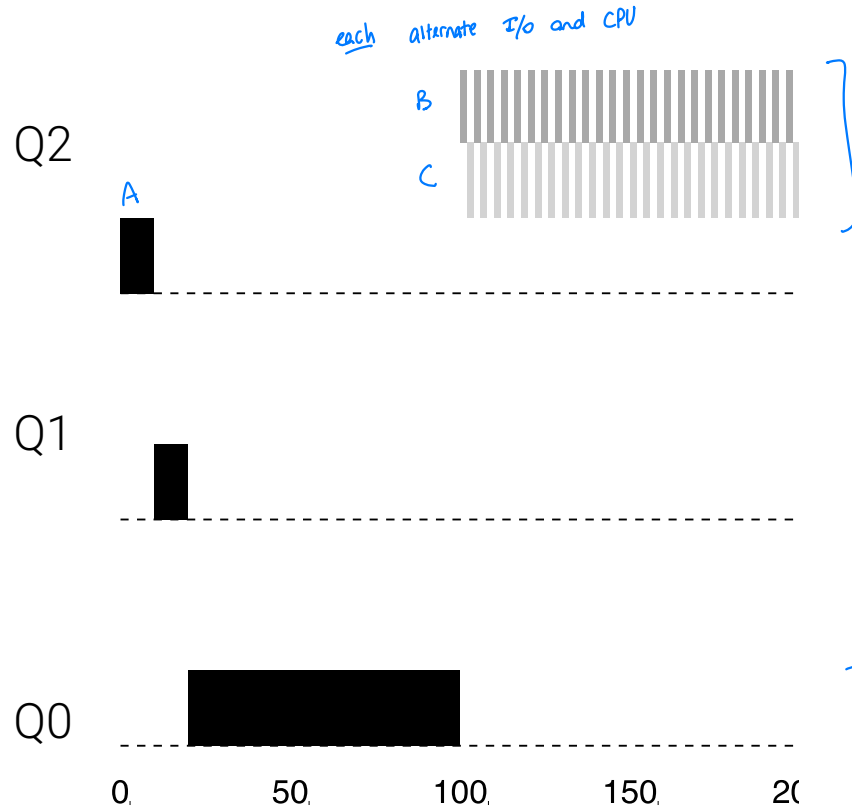
ONE LONG JOB



INTERACTIVE PROCESS JOINS



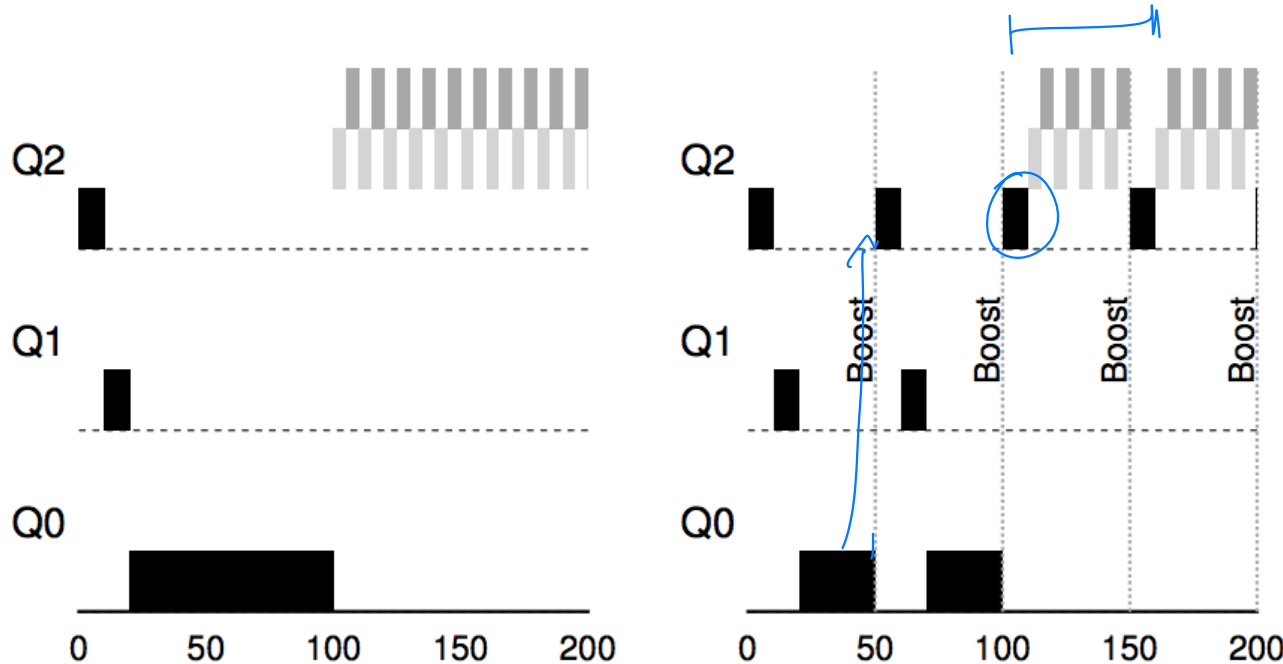
MLFQ PROBLEMS?



What is the problem with this schedule?



AVOIDING STARVATION



time slice per queue

Rule 5: After some time period S , move all the jobs in the system to the topmost queue.

Lottery Scheduling

MLFQ is not fair – shorter burst gets more CPU time

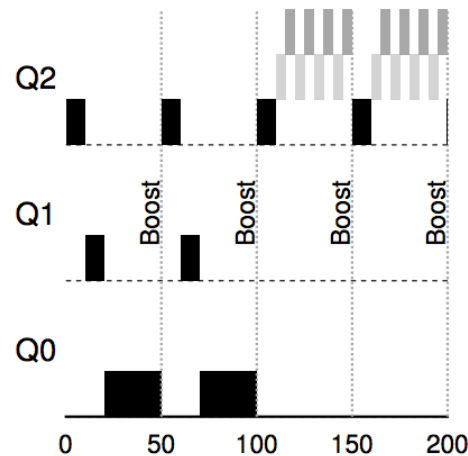
Goal: proportional (fair) share

- 3 users; each get 1/3rd of CPU, no matter how long they run for

Approach:

- give processes lottery tickets
- whoever wins runs
- higher priority => more tickets

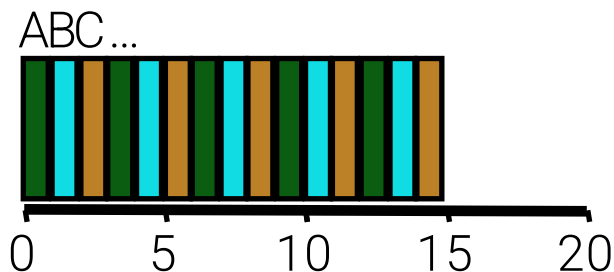
Amazingly simple to implement



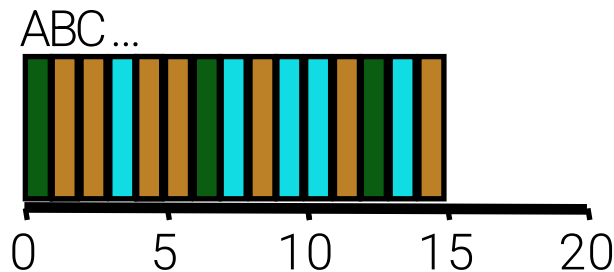
Lottery Example

JOB	Tickets
A	10
B	20
C	30

RR



Lottery



Lotteries control proportion of CPU

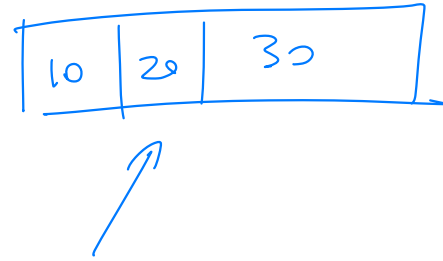
More tickets, higher proportion

When job runs is random

May win multiple lotteries in a row or lose many

Lottery Code

```
int counter = 0;
int winner = getRandom(0, totaltickets);
node_t *current = head;
while (current) {
    counter += current->tickets;
    if (counter > winner) break;
    current = current->next;
}
// current is the winner
```

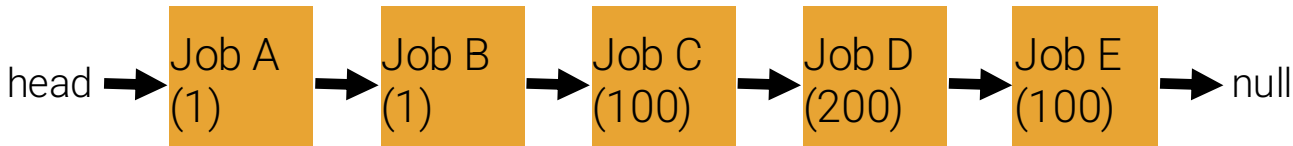


Lottery example

```
int counter = 0;  
int winner = getRandom(0, totaltickets);  
node_t *current = head;  
while (current) {  
    counter += current->tickets;  
    if (counter > winner) break;  
    current = current->next;  
}  
// current gets to run
```

Who runs if **winner** is:

50
350
0



SUMMARY

Scheduling Policies *FIFO, SJF, SCTF, RR, MLFQ, Lottery*

Understand *workload characteristics* like arrival, CPU, I/O

Scope out goals, *metrics* (turnaround time, response time)

Approach

Trade-offs based on goals, metrics (RR vs. SCTF)

Past behavior is good predictor of future behavior?